

Deep Learning Assignment: CIFAR-10 Image Classification Using CNN

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout,
Flatten, Dense, BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau
```

Task 1: Data Exploration and Preparation

```
# Load dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-
python.tar.gz
170498071/170498071 ————— 3s 0us/step

# Display 5 sample images
class_names =
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'shi
p', 'truck']

plt.figure(figsize=(10,2))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
plt.show()
```



```
# Print shapes and unique labels
print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)
print("Unique labels:", np.unique(y_train))
```

```

Training data shape: (50000, 32, 32, 3)
Test data shape: (10000, 32, 32, 3)
Unique labels: [0 1 2 3 4 5 6 7 8 9]

# Normalize pixel values
x_train, x_test = x_train / 255.0, x_test / 255.0

# One-hot encoding of labels
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout,
Flatten, Dense
from tensorflow.keras.callbacks import EarlyStopping

```

Task 2: Build and Train a CNN Model

```

# Building CNN model
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    MaxPooling2D((2,2)),
    Dropout(0.3),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Dropout(0.3),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Dropout(0.4),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

/usr/local/lib/python3.11/dist-packages/keras/src/layers/
convolutional/base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

# Compiling the model
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.summary()

```

Model: "sequential"

Layer (type) Param #	Output Shape	
conv2d (Conv2D) 896	(None, 30, 30, 32)	
max_pooling2d (MaxPooling2D) 0	(None, 15, 15, 32)	
dropout (Dropout) 0	(None, 15, 15, 32)	
conv2d_1 (Conv2D) 18,496	(None, 13, 13, 64)	
max_pooling2d_1 (MaxPooling2D) 0	(None, 6, 6, 64)	
dropout_1 (Dropout) 0	(None, 6, 6, 64)	
conv2d_2 (Conv2D) 73,856	(None, 4, 4, 128)	
max_pooling2d_2 (MaxPooling2D) 0	(None, 2, 2, 128)	
dropout_2 (Dropout) 0	(None, 2, 2, 128)	
flatten (Flatten) 0	(None, 512)	
dense (Dense) 65,664	(None, 128)	

dropout_3 (Dropout)	(None, 128)	
0		
dense_1 (Dense)	(None, 10)	
1,290		

Total params: 160,202 (625.79 KB)

Trainable params: 160,202 (625.79 KB)

Non-trainable params: 0 (0.00 B)

Training adam data

```
history = model.fit(x_train,
y_train_cat, epochs=15, batch_size=64, validation_split=0.2
)
```

Epoch 1/15

625/625 ————— 60s 95ms/step - accuracy: 0.6577 - loss: 0.9830 - val_accuracy: 0.7109 - val_loss: 0.8250

Epoch 2/15

625/625 ————— 80s 93ms/step - accuracy: 0.6641 - loss: 0.9641 - val_accuracy: 0.7412 - val_loss: 0.7445

Epoch 3/15

625/625 ————— 84s 96ms/step - accuracy: 0.6683 - loss: 0.9625 - val_accuracy: 0.7528 - val_loss: 0.7278

Epoch 4/15

625/625 ————— 60s 96ms/step - accuracy: 0.6719 - loss: 0.9464 - val_accuracy: 0.7281 - val_loss: 0.7650

Epoch 5/15

625/625 ————— 82s 96ms/step - accuracy: 0.6705 - loss: 0.9438 - val_accuracy: 0.7533 - val_loss: 0.7307

Epoch 6/15

625/625 ————— 82s 96ms/step - accuracy: 0.6750 - loss: 0.9382 - val_accuracy: 0.7333 - val_loss: 0.7621

Epoch 7/15

625/625 ————— 59s 95ms/step - accuracy: 0.6771 - loss: 0.9259 - val_accuracy: 0.7416 - val_loss: 0.7424

Epoch 8/15

625/625 ————— 60s 96ms/step - accuracy: 0.6788 - loss: 0.9230 - val_accuracy: 0.7515 - val_loss: 0.7216

Epoch 9/15

625/625 ————— 82s 95ms/step - accuracy: 0.6798 - loss: 0.9259 - val_accuracy: 0.7402 - val_loss: 0.7497

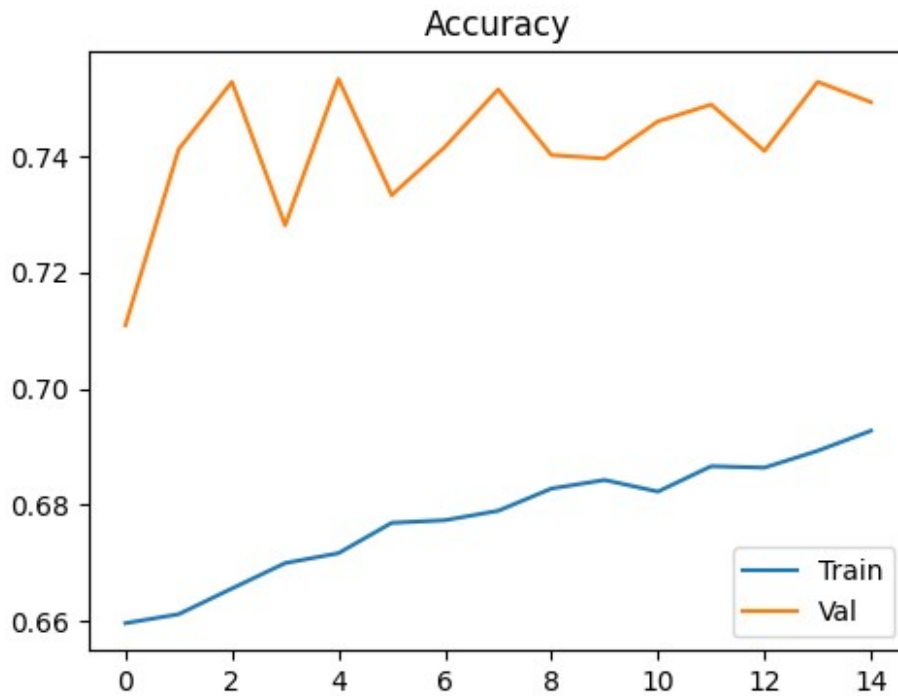
Epoch 10/15

```
625/625 _____ 60s 96ms/step - accuracy: 0.6817 - loss:
0.9207 - val_accuracy: 0.7396 - val_loss: 0.7539
Epoch 11/15
625/625 _____ 81s 95ms/step - accuracy: 0.6831 - loss:
0.9150 - val_accuracy: 0.7460 - val_loss: 0.7275
Epoch 12/15
625/625 _____ 81s 93ms/step - accuracy: 0.6879 - loss:
0.9058 - val_accuracy: 0.7489 - val_loss: 0.7391
Epoch 13/15
625/625 _____ 83s 94ms/step - accuracy: 0.6898 - loss:
0.8938 - val_accuracy: 0.7409 - val_loss: 0.7397
Epoch 14/15
625/625 _____ 59s 94ms/step - accuracy: 0.6914 - loss:
0.8956 - val_accuracy: 0.7528 - val_loss: 0.7028
Epoch 15/15
625/625 _____ 82s 94ms/step - accuracy: 0.6937 - loss:
0.8882 - val_accuracy: 0.7493 - val_loss: 0.7178
```

The Accuracy is 69% which is not bad but is not ideally good for predictions. This mainly happened because of the model's complex nature.

```
# Plot accuracy and loss
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title('Accuracy')
plt.legend()

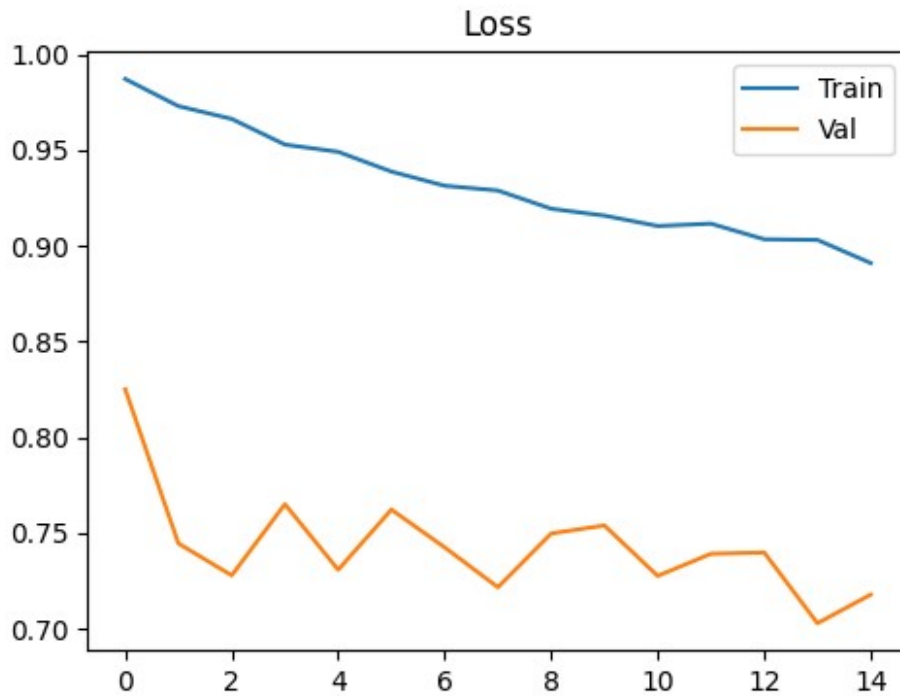
<matplotlib.legend.Legend at 0x7ce13b98e8d0>
```



Insights from the line plot

- Val Accuracy starts around 70% and reaches ~75%.
- Training Accuracy lags behind, starting at ~66% and slowly rising to ~69%.
- This gap suggests the model generalizes well, but isn't learning the training data as effectively as expected.
- Model is NOT overfitting

```
plt.figure(figsize=(12,4))
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.title('Loss')
plt.legend()
plt.show()
```



- Both Training and Validation Loss Decrease
- No Overfitting

Task 3: Evaluate the Model

```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Evaluate on test set
test_loss, test_acc = model.evaluate(x_test, y_test_cat)
print("Test accuracy:", round(test_acc, 4))
```

313/313 ————— 4s 13ms/step - accuracy: 0.7430 - loss: 0.7546
Test accuracy: 0.7385

The testing accuracy is 73% only, mainly because the model was too complex to interpret, even though trying with the Data Augmentation the model accuracy couldn't increase more than this.

```
# Confusion matrix and report
y_pred = model.predict(x_test)
y_pred_labels = np.argmax(y_pred, axis=1)
y_true_labels = y_test.flatten()
```

313/313 ————— 4s 12ms/step

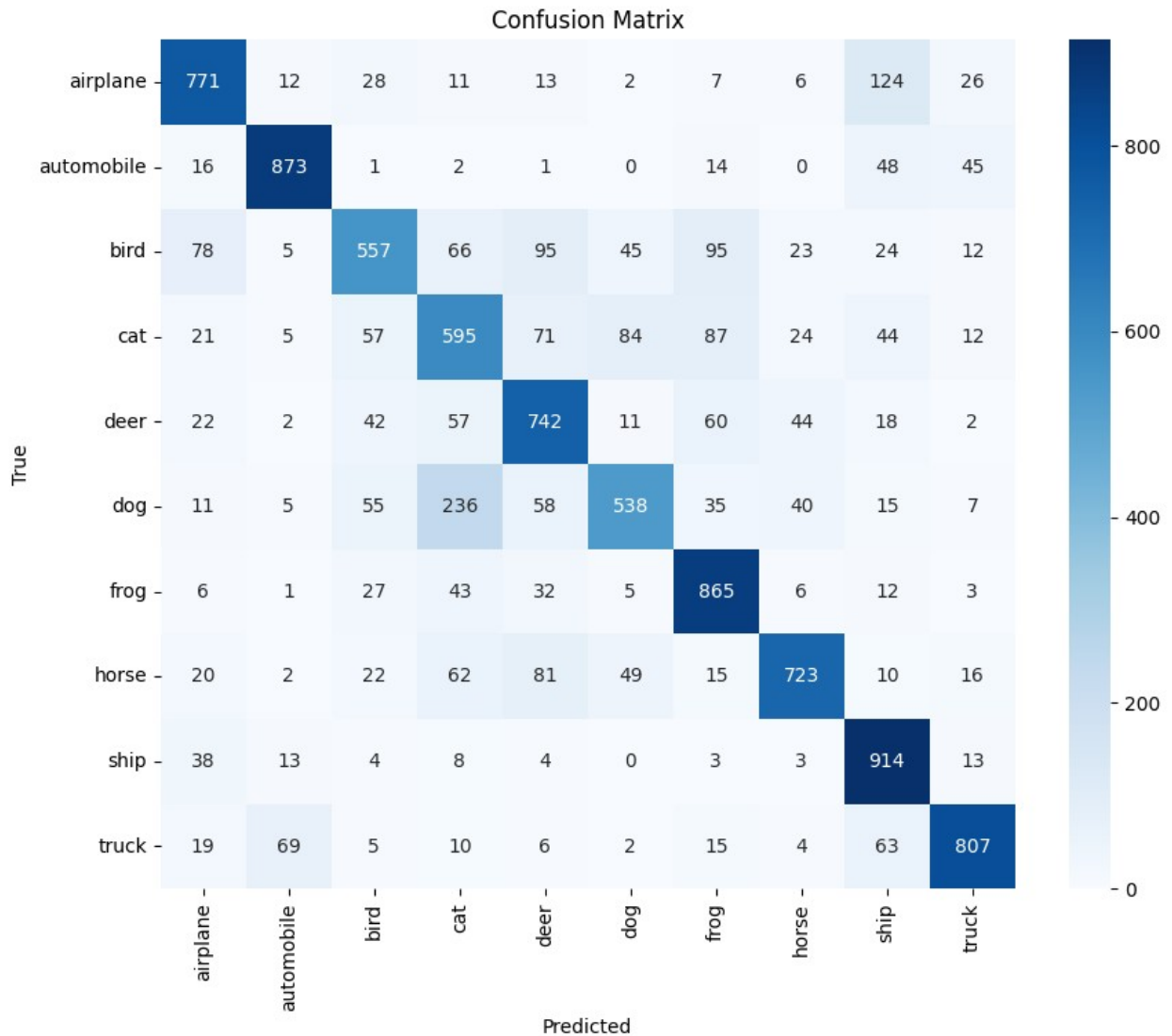
```
print("\nClassification Report:")
print(classification_report(y_true_labels, y_pred_labels,
target_names=class_names))
```

Classification Report:

	precision	recall	f1-score	support
airplane	0.77	0.77	0.77	1000
automobile	0.88	0.87	0.88	1000
bird	0.70	0.56	0.62	1000
cat	0.55	0.59	0.57	1000
deer	0.67	0.74	0.71	1000
dog	0.73	0.54	0.62	1000
frog	0.72	0.86	0.79	1000
horse	0.83	0.72	0.77	1000
ship	0.72	0.91	0.80	1000
truck	0.86	0.81	0.83	1000
accuracy			0.74	10000
macro avg	0.74	0.74	0.74	10000
weighted avg	0.74	0.74	0.74	10000

The accuracy for Adam default model is 74% which is overall good and the best accuracy compared to the other optimizer model

```
# Confusion Matrix
cm = confusion_matrix(y_true_labels, y_pred_labels)
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_names,
yticklabels=class_names, cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

```
# Displaying 3 correct and 3 incorrect predictions
correct = np.where(y_pred_labels == y_true_labels)[0]
incorrect = np.where(y_pred_labels != y_true_labels)[0]

plt.figure(figsize=(12,4))
for i, idx in enumerate(correct[:3]):
    plt.subplot(2,3,i+1)
    plt.imshow(x_test[idx])
    plt.title(f"Correct: {class_names[y_true_labels[idx]]}")
    plt.axis('off')
```

Correct: cat



Correct: ship



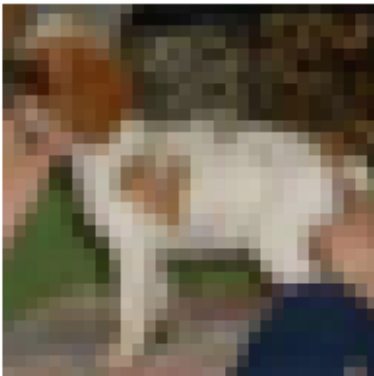
Correct: ship



Even though the accuracy was not that good but still the model predicted most of the images accurate.

```
for i, idx in enumerate(incorrect[:3]):
    plt.subplot(2,3,i+4)
    plt.imshow(x_test[idx])
    plt.title(f"Wrong: {class_names[y_pred_labels[idx]]}\nTrue:
{class_names[y_true_labels[idx]]}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

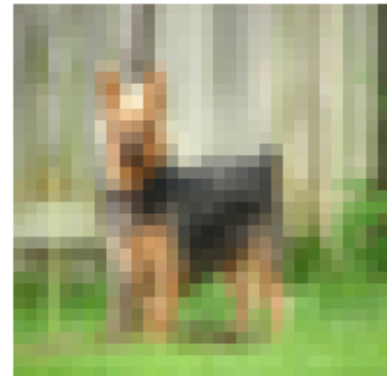
Wrong: cat
True: dog



Wrong: deer
True: horse



Wrong: deer
True: dog



Task 4: Experimentation with Model Improvements

```
from tensorflow.keras.optimizers import RMSprop, SGD

# Experimenting with different optimizer
model_rms = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    MaxPooling2D((2,2)), Dropout(0.3),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)), Dropout(0.3),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D((2,2)), Dropout(0.4),
    Flatten(), Dense(128, activation='relu'),
```

```

Dropout(0.5), Dense(10, activation='softmax')
])

/usr/local/lib/python3.11/dist-packages/keras/src/layers/
convolutional/base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

model_rms.compile(optimizer=RMSprop(),
loss='categorical_crossentropy', metrics=['accuracy'])

history_rms = model_rms.fit(x_train, y_train_cat,
epochs=20, batch_size=64, validation_split=0.2)

Epoch 1/20
625/625 _____ 65s 100ms/step - accuracy: 0.2161 - loss:
2.0859 - val_accuracy: 0.4195 - val_loss: 1.6217
Epoch 2/20
625/625 _____ 79s 96ms/step - accuracy: 0.4109 - loss:
1.6120 - val_accuracy: 0.4980 - val_loss: 1.4036
Epoch 3/20
625/625 _____ 87s 104ms/step - accuracy: 0.4807 - loss:
1.4487 - val_accuracy: 0.5575 - val_loss: 1.2589
Epoch 4/20
625/625 _____ 75s 93ms/step - accuracy: 0.5128 - loss:
1.3659 - val_accuracy: 0.5867 - val_loss: 1.1960
Epoch 5/20
625/625 _____ 84s 96ms/step - accuracy: 0.5425 - loss:
1.2975 - val_accuracy: 0.6226 - val_loss: 1.0706
Epoch 6/20
625/625 _____ 82s 96ms/step - accuracy: 0.5624 - loss:
1.2400 - val_accuracy: 0.6321 - val_loss: 1.0659
Epoch 7/20
625/625 _____ 82s 96ms/step - accuracy: 0.5858 - loss:
1.1807 - val_accuracy: 0.6427 - val_loss: 1.0149
Epoch 8/20
625/625 _____ 60s 96ms/step - accuracy: 0.5988 - loss:
1.1585 - val_accuracy: 0.6579 - val_loss: 0.9939
Epoch 9/20
625/625 _____ 82s 95ms/step - accuracy: 0.6029 - loss:
1.1458 - val_accuracy: 0.6431 - val_loss: 1.0115
Epoch 10/20
625/625 _____ 58s 93ms/step - accuracy: 0.6083 - loss:
1.1289 - val_accuracy: 0.6612 - val_loss: 0.9691
Epoch 11/20
625/625 _____ 85s 98ms/step - accuracy: 0.6174 - loss:
1.1210 - val_accuracy: 0.6790 - val_loss: 0.9364

```

```

Epoch 12/20
625/625 _____ 58s 93ms/step - accuracy: 0.6225 - loss:
1.1052 - val_accuracy: 0.6819 - val_loss: 0.9268
Epoch 13/20
625/625 _____ 83s 95ms/step - accuracy: 0.6234 - loss:
1.1021 - val_accuracy: 0.6754 - val_loss: 0.9268
Epoch 14/20
625/625 _____ 82s 96ms/step - accuracy: 0.6265 - loss:
1.0966 - val_accuracy: 0.6910 - val_loss: 0.9545
Epoch 15/20
625/625 _____ 61s 97ms/step - accuracy: 0.6264 - loss:
1.0866 - val_accuracy: 0.6730 - val_loss: 0.9570
Epoch 16/20
625/625 _____ 81s 96ms/step - accuracy: 0.6332 - loss:
1.0760 - val_accuracy: 0.6936 - val_loss: 0.9343
Epoch 17/20
625/625 _____ 83s 97ms/step - accuracy: 0.6329 - loss:
1.0898 - val_accuracy: 0.6998 - val_loss: 0.9436
Epoch 18/20
625/625 _____ 60s 95ms/step - accuracy: 0.6366 - loss:
1.0786 - val_accuracy: 0.6618 - val_loss: 0.9794
Epoch 19/20
625/625 _____ 60s 96ms/step - accuracy: 0.6355 - loss:
1.0847 - val_accuracy: 0.6803 - val_loss: 1.0880
Epoch 20/20
625/625 _____ 61s 98ms/step - accuracy: 0.6403 - loss:
1.0798 - val_accuracy: 0.6785 - val_loss: 0.9469

```

The rms optimizer accuracy is only 64% which will even low compared to the default model's accuracy

SGD Model Optimizer

```

model_sgd = tf.keras.models.clone_model(model)

model_sgd.compile(optimizer='SGD',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

history_sgd = model_sgd.fit(x_train, y_train, epochs=15,
                           validation_split=0.2, batch_size=64)

Epoch 1/15
625/625 _____ 66s 106ms/step - accuracy: 0.1256 - loss:
2.2881 - val_accuracy: 0.1934 - val_loss: 2.1975
Epoch 2/15
625/625 _____ 76s 96ms/step - accuracy: 0.1930 - loss:
2.1540 - val_accuracy: 0.2255 - val_loss: 2.1110
Epoch 3/15
625/625 _____ 61s 98ms/step - accuracy: 0.2099 - loss:

```

```

2.0880 - val_accuracy: 0.2400 - val_loss: 2.0389
Epoch 4/15
625/625 ━━━━━━━━━━━ 81s 97ms/step - accuracy: 0.2198 - loss:
2.0449 - val_accuracy: 0.2621 - val_loss: 1.9857
Epoch 5/15
625/625 ━━━━━━━━━━━ 81s 96ms/step - accuracy: 0.2509 - loss:
1.9747 - val_accuracy: 0.3202 - val_loss: 1.8845
Epoch 6/15
625/625 ━━━━━━━━━━━ 60s 95ms/step - accuracy: 0.2868 - loss:
1.9017 - val_accuracy: 0.2973 - val_loss: 1.9210
Epoch 7/15
625/625 ━━━━━━━━━━━ 84s 98ms/step - accuracy: 0.3042 - loss:
1.8435 - val_accuracy: 0.3746 - val_loss: 1.7637
Epoch 8/15
625/625 ━━━━━━━━━━━ 81s 96ms/step - accuracy: 0.3284 - loss:
1.7933 - val_accuracy: 0.3953 - val_loss: 1.7262
Epoch 9/15
625/625 ━━━━━━━━━━━ 59s 94ms/step - accuracy: 0.3547 - loss:
1.7403 - val_accuracy: 0.4020 - val_loss: 1.6675
Epoch 10/15
625/625 ━━━━━━━━━━━ 84s 97ms/step - accuracy: 0.3706 - loss:
1.6977 - val_accuracy: 0.4296 - val_loss: 1.6093
Epoch 11/15
625/625 ━━━━━━━━━━━ 81s 95ms/step - accuracy: 0.3783 - loss:
1.6724 - val_accuracy: 0.4244 - val_loss: 1.6142
Epoch 12/15
625/625 ━━━━━━━━━━━ 61s 97ms/step - accuracy: 0.3933 - loss:
1.6477 - val_accuracy: 0.4421 - val_loss: 1.5498
Epoch 13/15
625/625 ━━━━━━━━━━━ 83s 98ms/step - accuracy: 0.4072 - loss:
1.6130 - val_accuracy: 0.4572 - val_loss: 1.5274
Epoch 14/15
625/625 ━━━━━━━━━━━ 79s 94ms/step - accuracy: 0.4207 - loss:
1.5854 - val_accuracy: 0.4772 - val_loss: 1.5013
Epoch 15/15
625/625 ━━━━━━━━━━━ 83s 96ms/step - accuracy: 0.4208 - loss:
1.5778 - val_accuracy: 0.4770 - val_loss: 1.4737

```

The SGM accuracy is the least among all other models.

```

# Accuracy from each optimizer
adam_acc = history.history['val_accuracy'][-1]
sgd_acc = history_sgd.history['val_accuracy'][-1]
rmsprop_acc = history_rms.history['val_accuracy'][-1]

# Printing performance comparison
print("\n Optimizer Performance Comparison:")
print(f"{'Optimizer':<10} | {'Val Accuracy (%)}'")
print("-" * 30)

```

```
print(f"{'Adam':<10} | {adam_acc * 100:.2f}")
print(f"{'SGD':<10} | {sgd_acc * 100:.2f}")
print(f"{'RMSprop':<10} | {rmsprop_acc * 100:.2f}")
```

□ Optimizer Performance Comparison:

Optimizer | Val Accuracy (%)

Adam	74.93
SGD	47.70
RMSprop	67.85

Overall Adam optimizer is the best choice for this prediction.